Practical 2

Csv to horus

```
import csv
import json
from google.colab import files

#step1: create a sample csv form
csv_file = "students.csv"
students = [
    ["ID", "Name", "Age", "Course"],
    [1, "Rahul", 20, "BCA"],
    [2, "Priya", 21, "BSc IT"],
    [3, "Amit", 22, "BCA"],
    [4, "Sneha", 20, "BSc CS"]
]
with open(csv_file, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(students)
print("Sample CSV file created", csv_file)
#step2 : convert csv to HOURS format
def csv_to_hours(input_file, output_file):
    records =[]
    with open(input_file, "r") as f:
        reader = csv.DictReader(f)
        for row in reader:
            records.append({
                "type":"record",
                "data": dict(row)
            })
    hours = {
        "horus_version":"1.0",
        "source_format": "CSV",
        "records":records
    }


  #save hours outpout
    with open(output_file,"w",encoding="utf-8") as f:
      json.dump(hours, f, indent = 4)
    return hours
#step 3: perform converstion

output_file = "students_horus.json"
horus_data = csv_to_hours(csv_file, output_file)
#step 4: display hous output

print("\n========Hours output========")
```

```
print(json.dumps(horus_data, indent=4))
#step 5: download the horus file
files.download(output_file)
```

Xml to horus

```
import xml.etree.ElementTree as ET
import json
from google.colab import files

#step1: create a sample xml file
xml_file="students.xml"
xml_data = """<?xml version="1.0" encoding="UTF-8"?>
<students>
    <student>
        <id>1</id>
        <name>Rahul</name>
        <age>20</age>
        <course>BCA</course>
    </student>
    <student>
    <id>2</id>
    <name>Priya</name>
    <age>21</age>
    <course>BCA</course>
</student>

<student>
    <id>3</id>
    <name>Amit</name>
    <age>22</age>
    <course>Data Science</course>
</student>

<student>
    <id>4</id>
    <name>Sneha</name>
    <age>20</age>
    <course>Computer Science</course>
</student>

<student>
    <id>5</id>
    <name>Arjun</name>
    <age>23</age>
    <course>Artificial Intelligence</course>
    </student>
</students>
```

```
    """

with open(xml_file, "w", encoding="utf-8") as f:
    f.write(xml_data)
print(f"Sample XML file create", xml_file)
#step 2: convert XML elements into python dictionary

def xml_to_dict(elements):
  # If the elements has no child elements
  if(len(elements)) == 0:
    return elements.text.strip() if elements.text else ""

  data = {}
  for child in elements:
    child_data = xml_to_dict(child)
    #Handle repeated elements
    if child.tag in data:
      if not isinstance(data[child.tag], list):
        data[child.tag] = [data[child.tag]]
      data[child.tag].append(child_data)
    else:
      data[child.tag] = child_data
  return data
#sTEP 3: convert xml to horus
def xml_to_horus(input_file, output_file):
  tree = ET.parse(input_file)
  root = tree.getroot()
  data = xml_to_dict(root)
  horus = {
      "horus_version":"1.0",
      "source_format":"XML",
      "root":root.tag,
      "data":data
  }
  #save horus output
  with open(output_file, "w", encoding="utf-8") as f:
    json.dump(horus, f, indent=4)
  return horus
# Step4 : perform conversion

output_file = "students_horus.xml.json"
horus_data = xml_to_horus(xml_file, output_file)
# step5: display hours output
print("\n========Hours output========")
print(json.dumps(horus_data, indent=4))
#step 6:Download Hours file
files.download(output_file)
```

Json to horus

```
import json
from google.colab import files
data = [
    {
        "id": 101,
        "name": "AbdulR",
        "department": "IT",
        "marks": 85
    },
    {
        "id": 102,
        "name": "Rahul",
        "department": "HR",
        "marks": 78
    },
    {
        "id": 103,
        "name": "Karan",
        "department": "Computer Science",
        "marks": 92
    },
    {
        "id": 104,
        "name": "Amit",
        "department": "IT",
        "marks": 88
    },
    {
        "id": 105,
        "name": "Sneha",
        "department": "Marketing",
        "marks": 81
    }
]

#step2: save data in json format
with open("students.json","w") as file:
  json.dump(data, file, indent=4)
print("JSON file created successfully")


#step3: rea json data
with open("students.json","r") as file:
  json_data = json.load(file)

print("Original json data:")
print(json.dumps(json_data, indent=4))
```

```
#Step 4: convert json data to sample HORUS text
def convert_to_horus(records):
  horus_records=[]

  for record in records:
    fields = []

    for key, value in record.items():
      fields.append(f"{key}={value}")

    horus_records.append(" | ".join(fields))
  return "\n".join(horus_records)

horus_data = convert_to_horus(json_data)

#step5 display converted data

print("\n Converted HORUS Data: ")
print(horus_data)
#step 6: save outpout to horus file
with open("students.hours", "w") as file:
  file.write(horus_data)
print("\nConversion completed successfully!")
print("Output saved as students.hours")

#step7: Download the output file
files.download("students.hours")
```

Mysql to horus

```
import pandas as pd
import sqlite3 as sq
from google.colab import files
#create a sample database
sInputFileName = "/content/utility.db"
sInputTable = "Country_Code"

conn = sq.connect(sInputFileName)
#step 2: create and populate the contrycode table
create_table = """
CREATE TABLE IF NOT EXISTS Country_Code (
    "Country" TEXT,
    "ISO-2-Code" TEXT,
    "ISO-3-Code" TEXT,
    "ISO-M49" TEXT
)
"""
```

```
conn.execute("DROP TABLE IF EXISTS Country_Code")
conn.execute(create_table)

country_data = [
    ("India", "IN", "IND", "356"),
    ("Australia", "AU", "AUS", "036"),
    ("Brazil", "BR", "BRA", "076"),
    ("Canada", "CA", "CAN", "124"),
    ("Japan", "JP", "JPN", "392"),
    ("United Kingdom", "GB", "GBR", "826"),
    ("United States", "US", "USA", "840")
]
conn.executemany(
    'INSERT INTO Country_Code VALUES (?, ?, ?, ?)',
    country_data
)

conn.commit()
#step 3: read data from database
sSQL = "SELECT * FROM Country_Code"
InputData = pd.read_sql(sSQL, conn)

print("Input Data Values")
print(InputData)

#step4: process the data
ProcessData = InputData.copy()

#remove unnecessary columns
ProcessData.drop("ISO-2-Code", axis=1, inplace=True)
ProcessData.drop("ISO-3-Code", axis=1, inplace=True)

#rename column
ProcessData.rename(columns={"Country":"CountryName","ISO-
M49":"CountryNumber"}, inplace=True)

#set country Number as the index
ProcessData.set_index("CountryNumber", inplace=True)

#Sort country Number in descending order
ProcessData.sort_values(by="CountryNumber", axis=0, ascending=False,
inplace=True)

print("\nProcessed Data values")
print(ProcessData)
#step 5: saved the converted output
OutputData = ProcessData
sOutputFileName = "/content/HORUS-MySQL-Country.csv"
```

```
OutputData.to_csv(sOutputFileName, index=False)
print("\n Database to HOURS - DONE")
print("Output file: ",sOutputFileName)
#step 6; close database connection
conn.close()

#download output file
files.download(sOutputFileName)
```

Jpeg to horus

```
import numpy as np
from PIL import Image
from google.colab import files
#step1: Upload the image
print("Upload an image file (JPG,PNG, etc:)")
uploaded = files.upload()

#step2: Read the uploaded image
filename = next(iter(uploaded))
image = Image.open(filename).convert("RGB")
print("\nImage loaded successfully!")
print("Image size: ", image.size)
print("Image mode: ", image.mode)
#step3: resize image for managable processing
image = image.resize((100,100))
#step 4: convert image into numerical array
image_array = np.array(image)
print("\nImage converted into numerical data")
print("Array shape", image_array.shape)
# step 5: Flattern image pixel values
pixel_data = image_array.reshape(-1,3)
#step 6: Save pixel data as CSV file
output_file = "picture_hours.csv"
np.savetxt(
    output_file,
    pixel_data,
    delimiter=",",
    fmt="%d",
    header="Red,Green,Blue",
    comments=""
)
#step 7: display sample output
print("\n First 10 pixel values: ")
print(pixel_data[:10])
print("\n Picture conversion completed!")
print("Output saved as:",output_file)
```

```
#step 8: Download output
files.download(output_file)
```

Video to horus

```
import os
import cv2
import shutil
import numpy as np
import pandas as pd
from google.colab import files
#step1: Uploaded video
print("Upload your video file")
uploaded = files.upload()
video_name = next(iter(uploaded))
video_path = "/content/"+video_name
#create temperory founder for frames
frame_folder = "/content/temp"
if os.path.exists(frame_folder):
  shutil.rmtree(frame_folder)
os.makedirs(frame_folder)
print("\nStart Movie to Frames")
#step 3: open video
vidcap = cv2.VideoCapture(video_path)
if not vidcap.isOpened():
  raise ValueError("Unable to open video, please upload a valid video
file")
fps = vidcap.get(cv2.CAP_PROP_FPS)

if fps <= 0:
  fps = 30
total_frames = int(vidcap.get(cv2.CAP_PROP_FRAME_COUNT))
duration = total_frames/fps
print("Video FPS ", fps)
print("Total Video frames: ", total_frames)
print("Duration in seconds: ", round(duration))

#Extract approximately one frame per second
frame_interval = max(1, int(fps))
count = 0
frame_number = 0
while True:
  success, image = vidcap.read()
  if not success:
    break
  if frame_number % frame_interval == 0:
    frame_path = os.path.join(
        frame_folder,
```

```
        f"frame-{count:04d}.jpg"

    )
    cv2.imwrite(frame_path,image)
    count += 1
    frame_number += 1
vidcap.release()
print("\nGenerated", count, "frames")
print("Movie to Frames - Done")
#step 4: Converted extracted frames to HOURS data
print("Start Frames to HORUS")
records = []
#Resize each frame to keep output manageable
for frame_file in sorted(os.listdir(frame_folder)):
  if not frame_file.lower().endswith((".jpg", ".jpeg", ".png")):
    continue

  frame_path = os.path.join(frame_folder, frame_file)
  image = cv2.imread(frame_path)
  if image is None:
    continue

  #Resize image
  image = cv2.resize(image, (32, 32))
  #convert BGR to RGB
  image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
  height, width, channels = image.shape

  #read every pixel
  for y in range(height):
    for x in range(width):
      red, green, blue = image[y, x]
      records.append({
          "XAxis": x,
          "YAxis": y,
          "Red": red,
          "Green": green,
          "Blue": blue,
          "Alpha": 255,
          "FileName": frame_file
      })
#step 5: create horus Dataframe
columns = [
    "XAxis",
    "YAxis",
    "Red",
    "Green",
    "Blue",
```

```
    "Alpha",
    "FileName"
]
ProcessedData = pd.DataFrame(records, columns = columns)
print("\nHORUS Data Preview")
print(ProcessedData.head(10))
print("\nRows:", ProcessData.shape[0])
print("Columns: ", ProcessData.shape[1])
#step 6: save horus outpout to csv
output_file = "/content/HORUS-Movie-Frame.csv"
ProcessData.to_csv(output_file, index=False)
print("\nMovie to HORUS - DONE")
print("Output saved as: ", output_file)
#step 7: Download output file
files.download(output_file)
```

Practical 5

5a: Perform error management on the given data using pandas package.

```
#error management

import pandas as pd
data = {
    'Name':['Jhon', 'Jane', None, 'Emily'],
    'Age':[25,None, 34, 40],
    'Salary':[5000,6000,None,8000]
}

df = pd.DataFrame(data)
print("Original data is")
print(df)


#DROP DUPLICATES

df = df.drop_duplicates()
df

df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
df["Name"] = df["Name"].fillna("unknown")

print("Cleaned Data")
print(df)
```

5b: Write a program to create the network routing diagram.

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

G.add_edges_from([
    ('Router A', 'Router B'),
    ('Router A', 'Router C'),
    ('Router B', 'Router D'),
    ('Router C', 'Router D')
])

nx.draw(G, with_labels=True,
        node_color="lightblue",
        node_size=2000)

plt.show()
```

5c: Write a python/R program to build acyclic graph.

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_edges_from([
    ("1", "3"),
    ("1", "2"),
    ("2", "4"),
    ("3", "5")
])

nx.draw(G, with_labels=True,
        node_size=1000)

plt.show()
```

5d

```
import pandas as pd

df_abdulrahman = pd.DataFrame({
    "product": ["TV", "PHONE", "LAPTOP", "TABLET"],
    "SALES": [120, 300, 200, 150]
})
```

```
billboard = df_abdulrahman.sort_values("SALES",
ascending=False).head(2)
print(billboard)
```

5e Write a python/R program to generate GML file from a csv file.

```
import pandas as pd
import networkx as nx

data = {
    "Source": ["A", "A", "B"],
    "Target": ["B", "C", "D"],
    "Signal": ["true", "true", "true"],
    "Intensity": ["7.0", "8.0", "9.0"]
}

df_abdulrahman = pd.DataFrame(data)
print(df_abdulrahman)

G = nx.from_pandas_edgelist(
    df_abdulrahman,
    source="Source",
    target="Target",
    edge_attr=["Signal", "Intensity"]
)

nx.write_gml(G, "network.gml")
print("GML File Created")
```

5f

```
import pandas as pd

df_abdulrahman = pd.DataFrame({
    "X": [10, 20, 30, 40],
    "Y": [15, 25, 35, 45]
})

warehouse_x = df_abdulrahman["X"].mean()
warehouse_y = df_abdulrahman["Y"].mean()

print(warehouse_x, warehouse_y)
```

5g: Write a program using clustering to determine new warehouse using the given data.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
df_abdulrahman = pd.DataFrame({
    "X": [10, 20, 30, 40],
    "Y": [15, 25, 35, 45],
    "Z": [15, 25, 35, 45]
})

kmeans = KMeans(n_clusters=2, random_state=0, n_init=10)
df_abdulrahman["Cluster"] = kmeans.fit_predict(df_abdulrahman)

print(df_abdulrahman)

plt.scatter(
    df_abdulrahman["X"],
    df_abdulrahman["Y"],
    c=df_abdulrahman["Cluster"],
    s=100
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker="x",
    s=200,
    c="red"
)

plt.show()
```

5h

```
import networkx as nx

G = nx.Graph()

G.add_weighted_edges_from([
    ("Mumbai", "Delhi", 5),
    ("Delhi", "Chennai", 7),
    ("Mumbai", "Chennai", 15)
])

route = nx.shortest_path(
    G, "Mumbai", "Chennai", weight="weight"
)

print(route)
```

5i

```
import pandas as pd

df = pd.DataFrame({
    "Packing": ["Box a", "Box b", "Box c"],
    "Cost": ["500", "300", "700"]
})

df["Cost"] = pd.to_numeric(df["Cost"])

best = df["Cost"].idxmin()
df = df.drop(best)

print(df)
```

5j

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_edges_from([
    ("Warehouse", "Area1"),
    ("Area1", "Area2"),
    ("Area2", "Customer")
])

nx.draw(G, with_labels=True,
        node_color="lightblue",
        node_size=2000,
        arrows=True)

plt.show()
```

5k

```
import pandas as pd

df_abdulrahman = pd.DataFrame({
    "Rate": [82.5, 83.0, 82.8, 84.1]
})

buy = df_abdulrahman["Rate"].min()
sell = df_abdulrahman["Rate"].max()

print("Buy Price:", buy)
print("Sell Price:", sell)
```

5l

```
#generate payroll
import pandas as pd

df_abdulrahman = pd.DataFrame({
    "Revenue": [10000, None, 12000],
    "Expense": [5000, 5000, None]
})

df_abdulrahman = df_abdulrahman.fillna(0)

df_abdulrahman["Profit"] = df_abdulrahman["Revenue"] -
df_abdulrahman["Expense"]

print(df_abdulrahman)
```

5m: Write python program to generate payroll from the given data.

```
import pandas as pd

df_abdulrahman = pd.DataFrame({
    "Employee": ["John", "Alice", "Bob"],
    "Basic": [30000, 35000, 4000]
})

df_abdulrahman["HRA"] = df_abdulrahman["Basic"] * 0.20
df_abdulrahman["DA"] = df_abdulrahman["Basic"] * 0.15

df_abdulrahman["Gross salary"] = (
    df_abdulrahman["Basic"] +
    df_abdulrahman["HRA"] +
    df_abdulrahman["DA"]
)

print(df_abdulrahman)
```

Practical 6: Write a program to build the time hub, links and satellites

```
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx

data_abdulrahman = {
    "CustomerID":[101,102,101,102,104,102],
    "OrderID":[5001,5002,5003,5004,5005,5006],
```

```
    "CustomerName":["Rakesh","Suresh","Mukesh","Ramesh","Naresh",'Rajes
h'],
    "Location":["India","America","Africa","Brazil","China","Nepal"],
    "Amount":[2500,1500,1000,3000,2500,6000],
    "LoadDate":pd.Timestamp.today().date()
}

df_abdulrahman = pd.DataFrame(data_abdulrahman)
df_abdulrahman
#Hub table
hub_customer_ar = df_abdulrahman[["CustomerID"]].drop_duplicates()
hub_customer_ar
#Link Table
link_customer_ar =
df_abdulrahman[["CustomerID","OrderID"]].drop_duplicates()
link_customer_ar
#Satellite table
satellite_customer_ar =
df_abdulrahman[["CustomerID","OrderID","CustomerName","Location"]]
satellite_customer_ar
#Now visualize the data

orders_abdulrahman =
df_abdulrahman.groupby("CustomerName")["OrderID"].count()
plt.figure(figsize=(7,4))
orders_abdulrahman.plot(kind="bar",color="orange")
plt.xlabel("CustomerNmae")
plt.ylabel("OrderID")
plt.xticks(rotation = 0)
plt.show
```

```
G_abdulrahman = nx.Graph()

for customer in hub_customer_ar["CustomerID"]:
  G_abdulrahman.add_node(customer,type="Customer")

for _, row in link_customer_ar.iterrows():
  customer_id = row["CustomerID"]
  order_id = row["OrderID"]
  G_abdulrahman.add_node(order_id,node_type="Order")
  G_abdulrahman.add_edge(customer_id,order_id)

  plt.figure(figsize=(10,6))
  pos_abdulrahman = nx.spring_layout(G_abdulrahman, seed = 42)
```

```
nx.draw(
    G_abdulrahman,
```

```
    pos_abdulrahman,
    with_labels=True,
    node_color="lightblue",
    node_size=600,
    font_size=9
)

plt.title("Customer_Order Relationship")
plt.show()
```

Practical 7: Write a program for transforming data.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df_abdulrahman = pd.read_csv("/content/titanic.csv")
df_abdulrahman
df = df_abdulrahman.drop_duplicates()
df
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

#checking missing values remaining
df[["Age","Embarked"]].isnull().sum()
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[0,12,18,35,60,100],
    labels=["Child","Teenager","Adult","Middle Age","Senior"],
)
df
#create family size
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df
#create fare category
df["FareCategory"] = pd.cut(
    df["Fare"],
    bins=[0,100,200,300,520],
    labels=["Low","Medium","High","Very High"],
)
df
#create isAlone
df["IsAlone"] = np.where(df["FamilySize"] == 1, "Yes","No")
df
survival = df.groupby("Pclass")["Survived"].mean()*100
```

```
plt.figure(figsize=(8,6))
survival.plot(kind="bar")
plt.title("Survival Rate by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Survival Rate (%)")
plt.grid(True)
plt.show()
```

Practical 8: Write a program for organizing data

```

import pandas as pd
import numpy as np

df_abdulrahman = pd.read_csv('/content/Social_Network_Ads.csv')

print('original shape', df_abdulrahman.shape)
df_abdulrahman.head()
#after loading the data do data Inspection
print('\n Information \n')
print(df_abdulrahman.info())

print('\n Missing vlaues \n')
print(df_abdulrahman.isna().sum())

print('\n Duplicate Values \n')
print(df_abdulrahman.duplicated().sum())


#Remove the duplicate records
df_abdulrahman.drop_duplicates(inplace=True)

#Handles missing values

for col in df_abdulrahman.columns:
  if df_abdulrahman[col].isna().sum() > 0:
    if df_abdulrahman[col].dtype == 'object':
      df_abdulrahman[col].fillna(df_abdulrahman[col].mode()[0], inplace
= True)
    else:
      df_abdulrahman[col].fillna(df_abdulrahman[col].mean(),inplace =
True)
#standardize gender values

df_abdulrahman['Gender'] =
df_abdulrahman['Gender'].str.strip().str.capitalize()
df_abdulrahman.head()
#rename columns
```

```
df_abdulrahman.rename(columns={
    "EstimatedSalary":"Salary",
    "Purchased":"Purchased_Status"
},inplace = True)

#remove impossible values
df_abdulrahman = df_abdulrahman[(df_abdulrahman['Age'] >=18)&
                                (df_abdulrahman['Age'] <=18)&
                                (df_abdulrahman['Salary'] > 0)
                                ]
#create salary bands using quantities
df_abdulrahman['Salary_Band'] = pd.qcut(df_abdulrahman['Salary'],q=4,
labels=['Low','Medium','High','Very High'])

#create age category

bins = [18,30,45,65]
labels = ['Young','Adult','Senior']
df_abdulrahman['Age_Category'] =
pd.cut(df_abdulrahman['Age'],bins=bins,labels=labels, right = False)

#Encode Gender
df_abdulrahman['Gender_Code'] =
df_abdulrahman['Gender'].replace({'Male':1,'Female':0})
df_abdulrahman.head()

#Normalize Salary
df_abdulrahman['Normalized_Salary'] = (df_abdulrahman['Salary'] -
df_abdulrahman['Salary'].min()) / (df_abdulrahman['Salary'].max() -
df_abdulrahman['Salary'].min())

#Salary Rank
df_abdulrahman['Salary_Rank'] =
df_abdulrahman['Salary'].rank(ascending=False, method = "dense")

#multi level sorting
df_abdulrahman.sort_values(
    by=["Age_Category",
    "Salary",
    "Purchased_Status"],
    ascending=[True,False,False],
    inplace=True
)

#Groups Statistics
summary = df_abdulrahman.groupby(["Gender","Age_Category"]).agg(
    Total_Customers=("User ID","count"),
```

```
    Average_Salary=("Salary","mean"),
    Total_Salary=("Salary","sum"),
    Maximum_Salary=("Salary","max"),
    Minimum_Salary=("Salary","min")
)
print("\n group summary")
print(summary)

#Pivot Table
pivot = pd.pivot_table(
    df_abdulrahman,
    index="Age_Category",
    columns="Gender",
    values="Salary",
    aggfunc=[
        np.mean,
        np.max,
        np.min

    ]
)
print("\n Pivot Table")
print(pivot,"\n\n")

#Multiindex dataset
organized = df_abdulrahman.set_index(["Gender","Age_Category"])
print("\n Organized Dataset \n")
print(organized.head())

#Detect outliers
Q1 = df_abdulrahman['Salary'].quantile(0.25)
Q3 = df_abdulrahman['Salary'].quantile(0.75)
IQR = Q3 - Q1
outliers = df_abdulrahman[(df_abdulrahman['Salary'] < (Q1 - 1.5 * IQR))
| (df_abdulrahman['Salary'] > (Q3 + 1.5 * IQR))]


#Export files
organized.to_csv("organized.csv")
summary.to_csv("group_summary.csv")
pivot.to_csv("pivot_table.csv")
outliers.to_csv("outliers.csv")
#
print("\n Files created successfully \n")
print("1. Organized Data: organized_data.csv")
print("2. Group Summary: group_summary.csv")
print("3. Pivot Table: pivot_table.csv")
print("4. Outliers: outliers.csv")
```

Practical 9: Write a program for generating data.

```
!pip install faker

import numpy as np
from faker import Faker
import random
import pandas as pd

fake = Faker()
Faker.seed(42)
import random
random.seed(42)
np.random.seed(42)
#number of records
n = 1000
#generate user id
user_id = np.arange(1001, 1001 + n)
#generate personal information
gender = np.random.choice(['Male', 'Female'], size = n, p = [0.55,
0.45])
age = np.random.randint(18, 61, size= n)

#Generate salary (depends on age)
salary = (age * 3500 + np.random.randint(15000, 80000, size =
n)).astype(int)
salary = np.clip(salary, 20000, 250000)
#Generate year of experience
experience = np.maximum(
    age - np.random.randint(-2,4, size = n),
    0
)
#Generate Education
education = np.random.choice(
    ['Diploma','Bachlor','Master','PhD'],
    size = n,
    p = [0.15,0.50,0.25,0.10]
)
#Generate city
city = np.random.choice(
    ['Mumbai','Delhi','Banglore','Hyderabad','Ahmedabad','Chennai','Kol
kata','Pune','Jaipur','Surat'],
    size = n
)
#Generate joining date
joining_date = [
```

```
    fake.date_between(start_date='-10y',end_date='today')
    for _ in range(n)
]
#Purchase probability
probability = (
    (salary/salary.max())*0.45
    +(experience/experience.max())*0.35
)
purchased = (probability >= 0.55).astype(int)
#build data frame
df= pd.DataFrame({
    'user_id':user_id,
    'Gender': gender,
    'Age': age,
    'Salary': salary,
    'Experience': experience,
    'Education': education,
    'City': city,
    'Joining Date': joining_date,
    'Purchased': purchased})
#Introduce Missing values
for column in ['Age', 'Salary','Education']:
  missing_rows = np.random.choice(df.index, 15, replace = False)
  df.loc[missing_rows, column] = np.nan


#Introduce duplicate values
duplicates = df.sample(20, random_state=42)
df = pd.concat([df, duplicates], ignore_index=True)

#shuffle dataset
df = df.sample(
    frac =  1,
    random_state = 42,
).reset_index(drop = True)
#Now your entire dataset is generated
#So you display the dataset
print("\n Dataset Shape")
print(df.shape)
print("\n Dataset Types")
print(df.dtypes)
print("\n First five rows")
print(df.head())
print("\n Last five rows")
print(df.tail())
print("\n Missing values")
print(df.isnull().sum())
print("\n Duplicated Records")
print(df.duplicated().sum())
```

```
print("\n Statistical summary")
print(df.describe(include='all'))

#save Dataset
df.to_csv("Generated_Employee_Dataset.csv", index = False)
print("\n Dataset Generated Sucessfully")
print("File Name : Generated_Employee_Dataset.csv")
print("File Location : /content/Generated_Employee_Dataset.csv")
```

Practical 10